

CS1CA Computer Systems Architecture

Instruction – machine language (2)

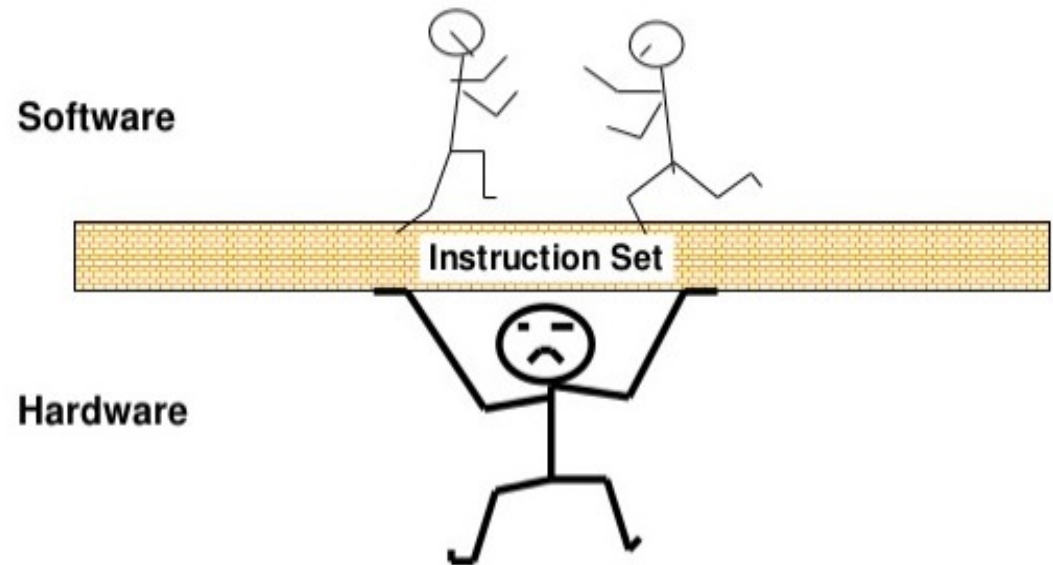
Week 5 lecture 2 of autumn semester 2025

Lectured by

Dr. Hong Wei

h.wei@reading.ac.uk

Room 133 in Polly Vacher



(Figure: courtesy of the Internet)

In this lecture

- The following contents will be delivered
 - Memory addressing
 - Operands in instruction sets
- The learning outcomes are listed below
 - To be able to use examples to explain the three addressing modes, i.e. displacement, immediate, and register indirect addressing
 - To be aware of the relation between data types/operands and instruction length, and the relation between memory addressing modes and instruction length (number of bits used to represent instructions).

Memory addressing

...	...
Address 3	11101000
Address 2	00000000
Address 1	10010111
Address 0	01101001

Note: In the lab practicals on the ARM architecture, there is a set of registers containing memory addresses where specific information is stored. For example, the SP register is to keep the stack pointer, which is the memory address of the beginning of the stack. Please apply the knowledge learned in this week to support the lab practicals.

Memory addressing: addressing modes

- Interpreting memory address:
Byte addressed with either *Little Endian* or *Big Endian*.
- Alignment to memory accesses (this will be seen in the ARM lab practicals.)
 - In many cases, a memory access is required to be aligned.
 - It refers to the arrangement of data in memory.
 - Let's see an example.

Example of memory alignment

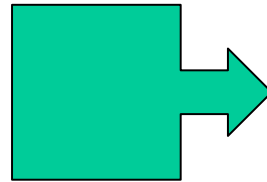
- A data structure is defined as

Struct data

```
{  
    char A, B;  
    int C;  
    char D, E;  
}
```

char: 1 byte; int: 4 bytes

sizeof (data) => 10 bytes



- In a memory, it is stored as below.

E	9
D	8
C4	7
C3	6
C1	5
C1	4
	3
	2
B	1
A	0

Redefine the data structure as

Struct data

```
{  
    char A, B, D, E;  
    int C;  
}
```

sizeof(data)
=> 8 bytes

Memory addressing: three modes

- Addressing modes: the ways architectures specify the address of an object (or operand) they will access.
- Three addressing modes:
 - Displacement addressing
 - Immediate addressing
 - Register indirect addressing
- These three addressing modes represent 75-99% of addressing modes used in a typical program.

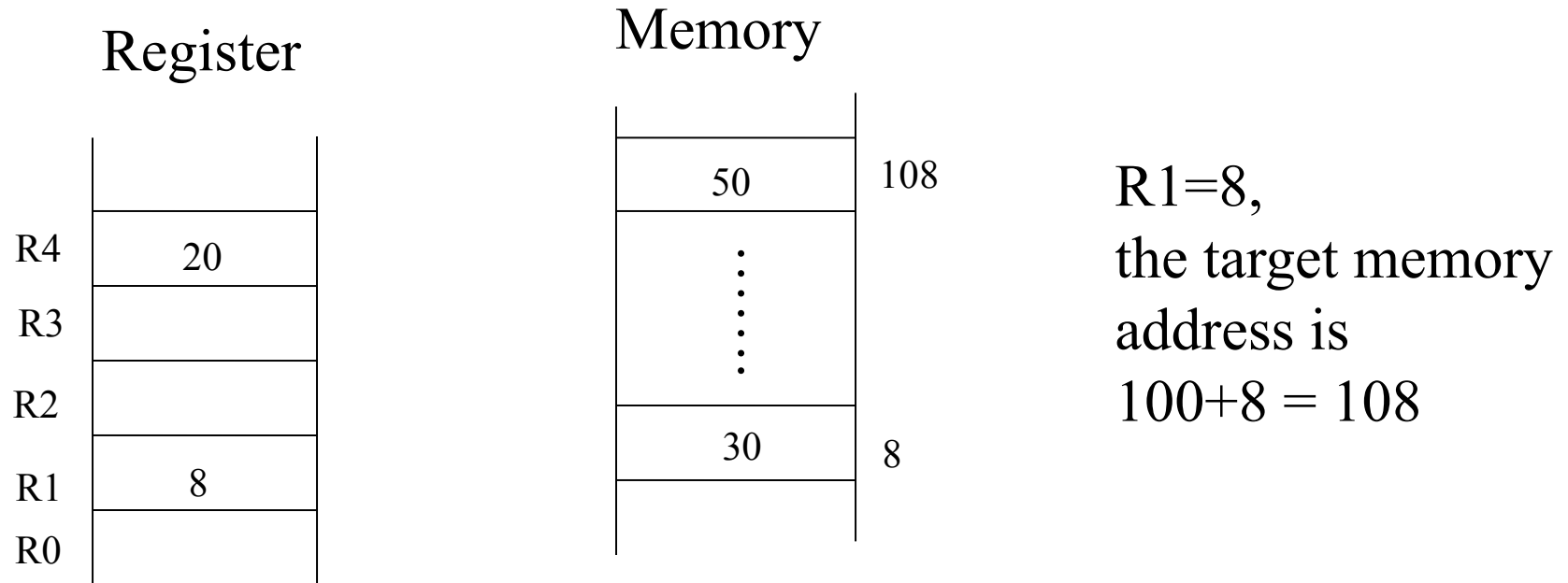
Examples: addressing modes

Addressing mode	Example instruction	Meaning	When used
Displacement	Add R4, 100(R1)	$\text{Regs}[\text{R4}] \leftarrow \text{Regs}[\text{R4}] + \text{Mem}[100 + \text{Regs}[\text{R1}]]$	Accessing local variables
Immediate	Add R4, #3	$\text{Regs}[\text{R4}] \leftarrow \text{Regs}[\text{R4}] + 3$	For constants
Register indirect	Add R4, (R1)	$\text{Regs}[\text{R4}] \leftarrow \text{Regs}[\text{R4}] + \text{Mem}[\text{Regs}[\text{R1}]]$	Accessing using a pointer or a computed address

The usage of various addressing modes is important in helping architects to choose what to include in the design.

Memory addressing mode: displacement

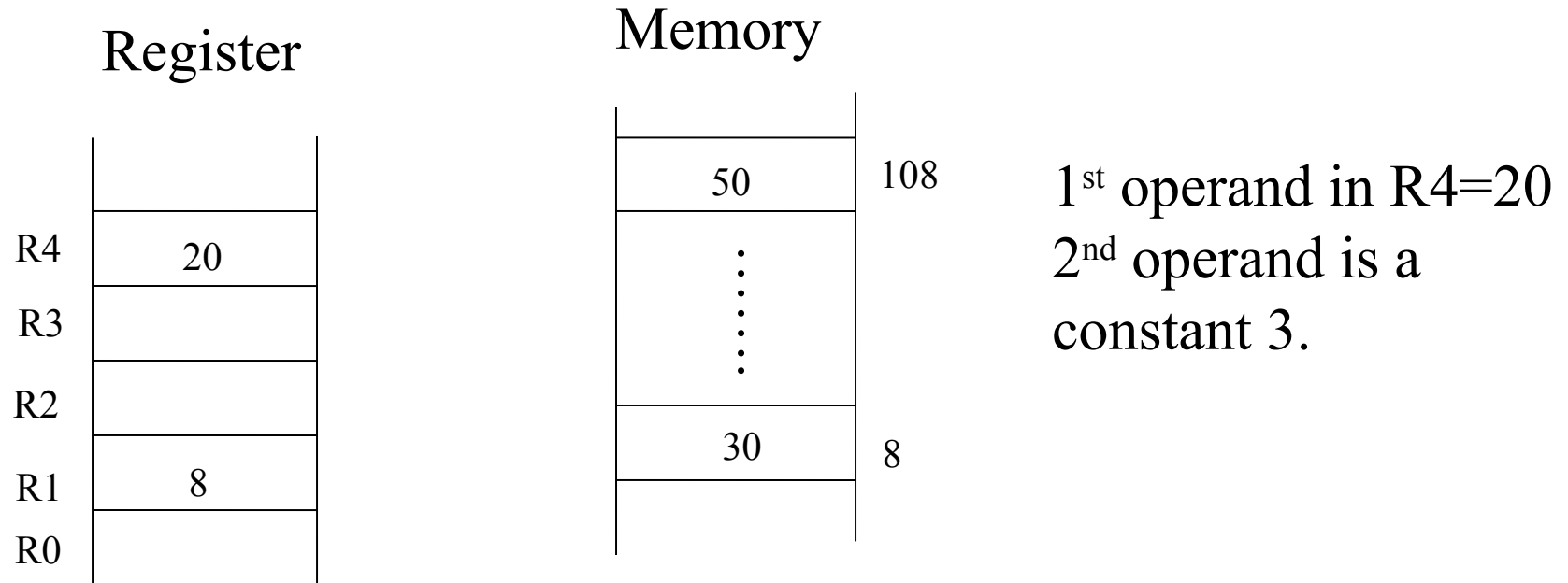
- *Example* Add R4, 100(R1)



Reg[R4]=20, Mem[108]=50,
After the instruction is executed, R4=70

Memory addressing mode: Immediate

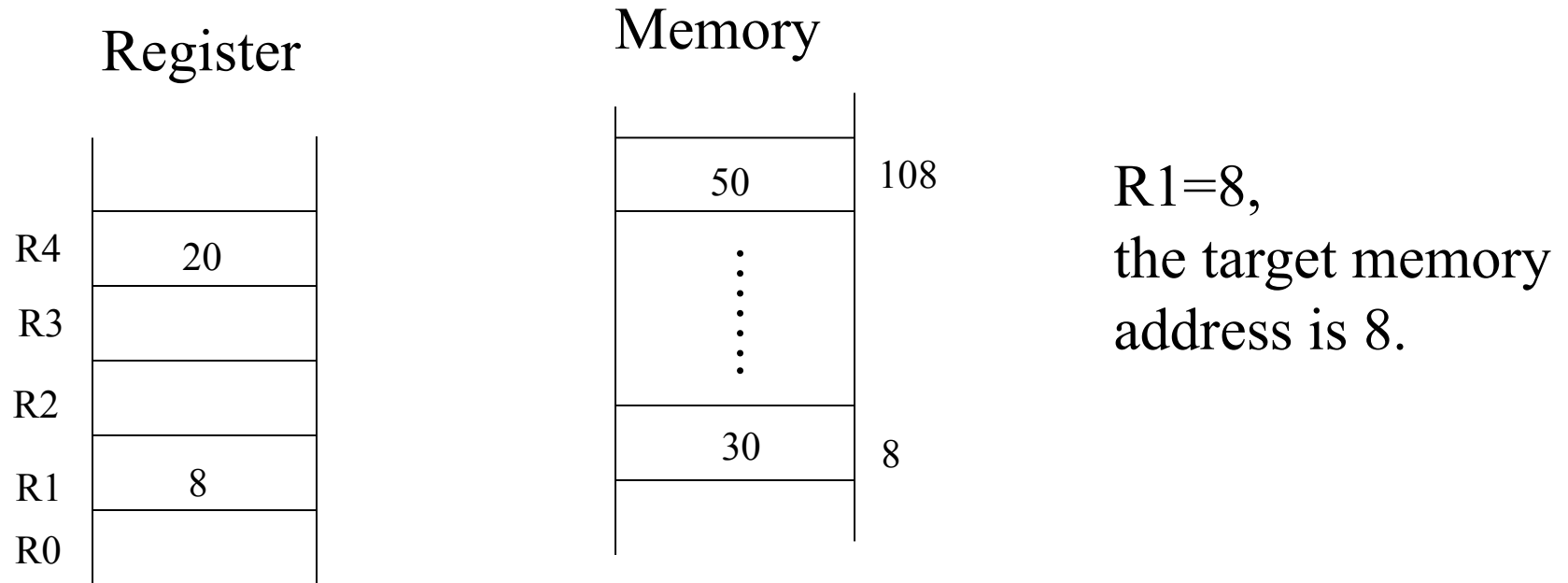
- *Example* Add R4, #3



After the instruction is executed, R4=23.
Where to find the constant 3?

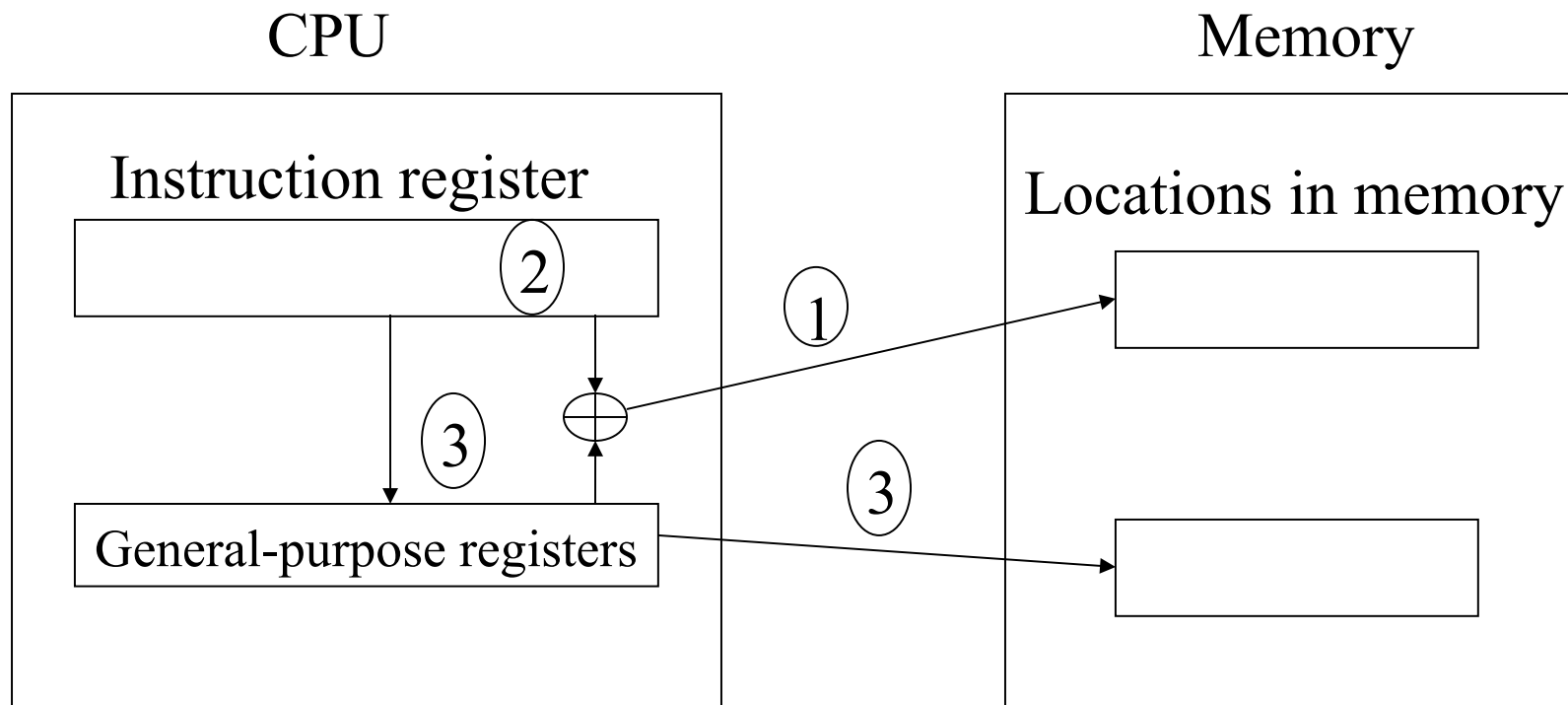
Memory addressing mode: register indirect

- *Example* Add R4, (R1)



Reg[R4]=20, Mem[8]=30,
After the instruction is executed, R4=50

Illustration of addressing modes



1. Displacement addressing; 2. Immediate addressing;
3. Register Indirect

Memory addressing: instruction length

- The size of the displacement field directly affects the instruction length.
- Displacement values are widely distributed.
- Normally displacement with 12-16 bits captures 75-99% of displacements.
- The size of immediate values also affects instruction lengths.
- The size of the immediate field with 8-16 bits captures 50-80% of the immediate.

Operands in instruction sets

- In desktop (laptop) and server architectures
 - **Type and size**: usually the type of an operand gives its size, such as
 - character (8 bits), integer (32 bits), SFP (32 bits),
byte (8 bits), half word (16 bits), word (32 bits),
double word (64 bits), DFP (64 bits),
- For media and **DSP** (**D**igital **S**ignal **P**rocessor)
 - 32-bit floating point for a vertex (3D graphics);
 - 32-bits with four 8-bit channels for pixels (2D images);
 - DSPs add *fixed point* to the data type, which are fractions between -1 and +1 as low-cost SFP.

End of the lecture

Questions and exercises

- Addressing modes
 - Why does an instruction need an addressing mode?
 - What are contents in register R3 when the following instructions are executed. At the beginning $R0=0$, $R3=30$.
 - Sub R3, 200(R0) - displacement addressing for subtraction
 - Sub R3, (R0) - register indirect addressing for subtraction
 - Sub R3, #10 - immediate addressing for subtractionwhere $\text{Mem}[0]=20$; $\text{Mem}[200]=100$. All data in decimal notation
- Operands are with different data types.
 - Why is it important to define a data type in your program?
 - How would a data type affect instruction length?